

# Model rollout to a serving fleet

front: what to ask, what to design · semi-guided, expect a twist · one problem

## 1 REQUIREMENTS 0–5 min · questions first, then sentences

- **Ask:** what is a worker: host, process or GPU? one loaded version per GPU? spare capacity for a rolling update?
- **Ask:** artifact size, immutable, version + checksum? origin outbound and per-worker link? fleet size and regions?
- **Ask:** must V1 keep serving during the rollout? what must be true before the first request reaches V2? what does "done" mean to the engineer?
- **A release engineer can:** publish a version · watch progress · pause, resume, roll back. **The system:** every targeted worker ends with a verified copy and serves it; the service stays available.
- **Technical challenges:** the origin link is the bottleneck · verify before serving · readiness = exact version · capacity-safe cohorts ( 20% headroom) · canary stop signals · resumable, idempotent, generation-checked reports.
- **Anchors:** 100 GB artifact · 100 GB GPU memory · 10 Gbit/s origin and per-worker · 10 workers then 1,000 in 3 regions · target 95% in 15 min.

## 2 CORE ENTITIES 5–9 min · records the failure probe needs

- **Artifact**(version, size, location, chunk list + hashes, checksum) — immutable
- **Rollout**(id, target group, desired version, **generation**, status running|paused|rolling back|done, cohort %, canary %)
- **Worker**(id, region, attempt/generation, actual version, state, heartbeat, bytes\_done, error)
- state: idle → downloading → verified → draining → loading → ready(version, generation) → serving
- Readiness identifies the exact version, never just a healthy machine. Every report carries the generation it was made under.

## 3 API 9–13 min · one endpoint per requirement

- **POST /releases** {version, checksum, manifest}
- **POST /rollouts** {version, targets, cohort\_pct, canary\_pct} · **/rollouts/{id}/pause | resume | rollback** (rollback = new generation, desired = V1)
- **POST /workers/{id}/report** {generation, state, version, bytes\_done, error} — older generation → ignored
- **GET /rollouts/{id}** → downloaded · verified · ready · serving, per region. The engineer acts on **serving**, never on downloaded.

## 4 DATA FLOW 13–20 min · one worker, end to end

- publish manifest + hashes → controller assigns generation G, picks a cohort inside the headroom →
- agent downloads by chunk, hash per chunk, resumable → whole-file checksum → controller: drain (in-flight finish, admission stops) →
- load into GPU memory + warm-up prompts → agent reports ready(V2, G) → router flips this worker → next cohort.
- **Before the first request hits V2:** verified, loaded, warmed, ready under the current generation, and enough V1 still serving.
- **Restart mid-download:** resume by verified chunk; a stale "done" with an old generation is discarded; duplicates change nothing.

## 5 HIGH-LEVEL DESIGN 20–33 min · the board, then the twist

- Row 1: engineer → rollout controller → artifact store (origin) → records. Row 2: one seed/cache per region. Row 3: regional worker swarms. Row 4: agent state machine · traffic router (exact-version ready only) · canary/health monitor.
- **Bandwidth first:** 100 GB = 800 Gbit → 80 s per copy at 10 Gbit/s · 1,000 copies via the origin = 80,000 s ≈ 22 h ; 950 ≈ 21 h. Direct download cannot make 15 min.
- **Redesign:** origin sends 3 copies ( 240 s ), one per region; fan out inside each region on links the origin does not share (cache tree or chunked peer swarm, every worker uploads while it downloads). Add load + warm-up to the clock.
- Trace the bytes: origin → regional seed → regional egress (next shared limit) → racks → workers. Feasible **only if**

## 6 DEEP DIVES 33–45 min · failure, rollback, recap

- **Load failures / ready-then-crash:** pull from routing; isolated vs systematic; stop signals = errors, latency, warm-up failures, capacity loss → pause.
- **Rollback:** route to healthy V1 capacity first; drain and reload V1 in cohorts; V1 on disk still needs load + warm-up, so not instant; record the rollback generation so a late V2 "ready" cannot reverse it.
- **Dashboard:** 95% downloaded but 60% can serve → act on serving capacity.
- **Peers forbidden across regions:** regional seeding and in-region fan-out unchanged; only the inter-region hop stays origin-fed.
- **Recap in one minute:** publish, distribute, verify, activate, route, recover. Guarantee: a request reaches only a verified, loaded, exact-version worker under the current generation. Remaining bottleneck: regional egress + load

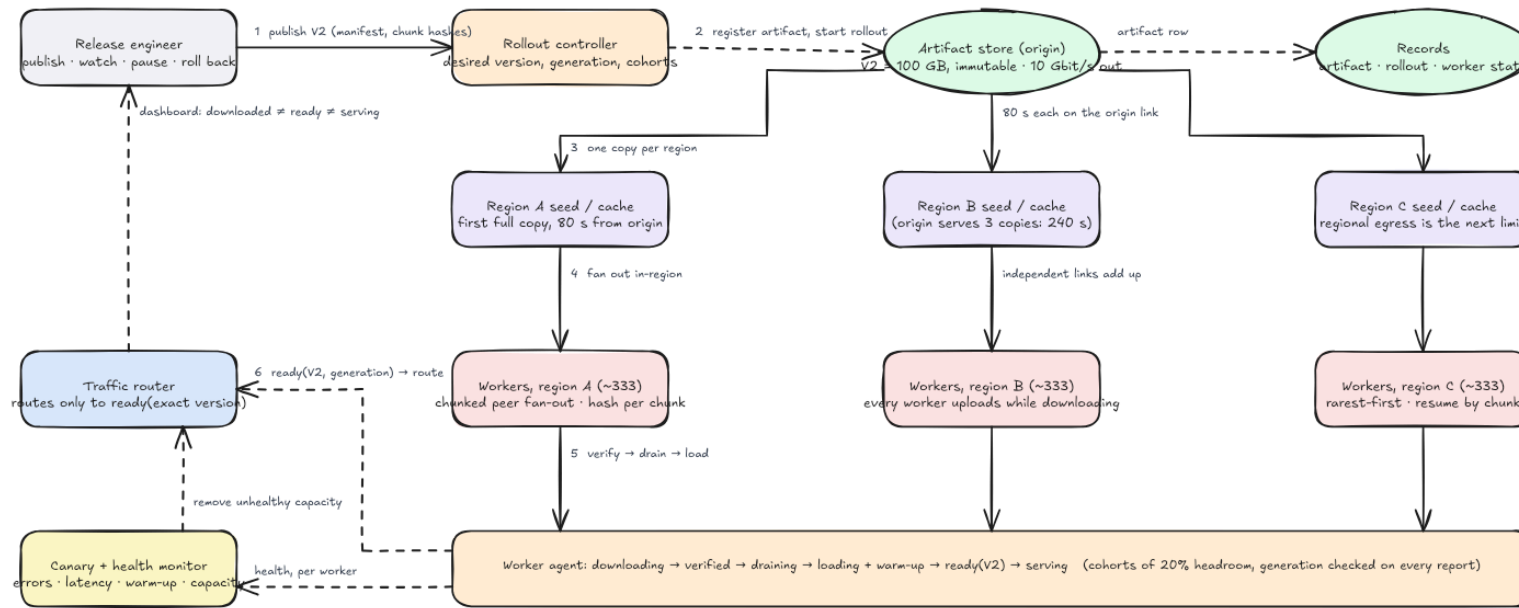
# The board at minute forty

back: the answer, as it should look on the shared screen  
regional egress and load time meet numbers you have not been given: say what you would measure.

time. Validate: measured fan-out throughput and warm-up on one cohort.

## Model rollout to a serving fleet, the answer

Publish once, stage one copy per region, fan out on independent links, verify every byte, drain in cohorts, load and warm, route only to exact-version-ready workers. Solid = bytes and requests, dashed = control.



Bandwidth budget first: 100 GB = 800 Gbit → 80 s per copy on a 10 Gbit/s link, 1,000 copies straight from the origin = 80,000 s = 22 h; even 950 copies = 21 h. Direct download cannot meet 15 minutes.  
Stage 3 copies (240 s of origin time), then fan out on links the origin does not share: tree or chunked swarm inside each region. Add loading and warm-up to the clock. Feasibility is conditional on regional egress you have not measured: say so.  
Recovery: a worker that restarts resumes by verified chunk; a stale "done" carrying an old generation is ignored; rollback is a new generation that routes to healthy V1 capacity first, then reloads V1 in cohorts (V1 on disk still needs load + warm-up).

Legend

- people, dashboards
- the service under design
- artifact store, records
- regional staging
- workers (data plane)
- traffic routing
- health, canary